

Lumina Chronica

Master Project Bible

Teil 6 – AI Development Instructions & Coding Guidelines

73. Purpose of AI Development Instructions

Dieses Kapitel definiert die Regeln für jede KI oder jeden Entwickler, der an Lumina Chronica arbeitet.

Die KI soll nicht einfach Code generieren.

Sie soll als:

- Softwarearchitekt
- Entwickler
- UI/UX-Designer
- Qualitätssicherer

arbeiten.

74. General AI Behavior Rules

74.1 Understand Before Implementing

Vor jeder größeren Änderung muss die KI:

1. Die bestehende Architektur verstehen.
 2. Die Auswirkungen der Änderung prüfen.
 3. Bestehende Funktionen berücksichtigen.
 4. Eine passende Implementierungsstrategie wählen.
-

Nicht erlaubt:

Idee bekommen

↓

direkt Code schreiben

↓

bestehende Architektur zerstören

Erwünscht:

Anforderung

↓

Analyse

↓

Plan

↓

Implementierung

↓

Test

↓

Dokumentation

74.2 Respect Existing Decisions

Bereits getroffene Entscheidungen dürfen nicht ohne Grund geändert werden.

Beispiele:

Nicht automatisch ändern:

- Blazor WebAssembly
 - Cloudflare Architektur
 - Monorepo
 - D1 Datenbank
 - R2 Storage
 - modulare Struktur
-

Wenn eine Änderung sinnvoll erscheint:

Die KI muss erklären:

- Warum?
 - Welche Vorteile?
 - Welche Nachteile?
 - Welche Auswirkungen?
-

74.3 Avoid Overengineering

Lumina Chronica soll professionell sein, aber nicht unnötig kompliziert.

Nicht automatisch hinzufügen:

- unnötige Frameworks
 - komplexe Microservices
 - überdimensionierte Systeme
 - Technologien ohne konkreten Nutzen
-

Prinzip:

Einfach starten, sauber erweitern.

75. Development Workflow

Jede größere Funktion folgt diesem Ablauf.

Phase 1 – Requirement Analysis

Die KI analysiert:

- Was soll erreicht werden?
 - Welche Benutzer profitieren?
 - Welche Systeme sind betroffen?
-

Beispiel:

Feature:

"Offline Bücher"

Analyse:

Betroffen:

Frontend

Reader

Storage

Synchronisation

Database

Phase 2 – Planning

Vor Code:

Erstellen:

- technische Lösung
- betroffene Dateien
- Datenänderungen
- mögliche Risiken

Beispiel:

Plan:

1. Neue OfflineBook Entity
2. Storage Service erweitern
3. Reader Cache hinzufügen
4. UI Button erstellen
5. Test hinzufügen

Phase 3 – Implementation

Beim Programmieren:

Beachten:

- bestehender Stil
 - Naming Convention
 - Architektur
 - Wiederverwendbarkeit
-

Phase 4 – Testing

Nach Änderungen prüfen:

- Funktioniert Feature?
 - Gibt es Fehler?
 - Werden bestehende Funktionen beeinflusst?
-

Phase 5 – Documentation

Neue wichtige Funktionen dokumentieren:

- Zweck
 - Verwendung
 - technische Entscheidungen
-

76. Coding Principles

76.1 Clean Code

Code soll:

- lesbar
- verständlich
- wartbar

sein.

Schlecht:

```
var x = GetData();
```

Besser:

```
var userLibrary = GetUserLibrary();
```

76.2 Naming

Namen müssen Bedeutung erklären.

Nicht:

```
BookManager2  
DataHandler  
Helper
```

Besser:

```
BookService  
  
ReadingProgressService  
  
StorageService
```

76.3 Single Responsibility

Eine Klasse soll eine klare Aufgabe haben.

Nicht:

```
BookService  
  
- Upload  
- Database  
- UI  
- Authentication  
- Statistics
```

Besser:

```
BookService  
UploadService  
StatisticsService
```

76.4 Reusable Components

UI-Komponenten sollen wiederverwendbar sein.

Beispiel:

Nicht:

```
HomeBookCard  
LibraryBookCard
```

Besser:

```
BookCard  
  
mit Varianten:  
  
Small  
  
Normal  
  
Large
```

77. Frontend Development Rules

Technology

Frontend:

- Blazor WebAssembly

- C#
 - Razor Components
-

77.1 Component Structure

Komponenten sollen:

- klein
- spezialisiert
- wiederverwendbar

sein.

Beispiel:

```
Components/
```

```
BookCard/
```

```
BookCard.razor
```

```
BookCard.razor.cs
```

```
BookCard.css
```

77.2 State Management

State soll bewusst verwaltet werden.

Nicht:

- globale Variablen
 - unkontrollierte Zustände
-

Bevorzugt:

- Services
 - Dependency Injection
 - klare Models
-

77.3 UI Rules

Jede Seite benötigt:

- Loading State
- Empty State
- Error State

Beispiel:

Nicht nur:

Keine Bücher

Sondern:

Deine Bibliothek ist noch leer.

Füge dein erstes Buch hinzu.

[Buch hinzufügen]

78. Backend Development Rules

78.1 API Design

APIs müssen:

- logisch
- dokumentiert
- konsistent

sein.

Beispiel:

Nicht:

```
/api/getBooksNow
```

Besser:

```
/api/books
```

78.2 Validation

Alle Benutzereingaben müssen geprüft werden.

Beispiele:

- Upload-Dateien
 - Texte
 - IDs
 - Berechtigungen
-

78.3 Security First

Immer prüfen:

```
Wer ist der Nutzer?
```

```
Hat er Zugriff?
```

```
Darf er diese Aktion durchführen?
```

79. Database Rules

79.1 Schema Changes

Datenbankänderungen benötigen:

- Migration
 - Dokumentation
 - Prüfung
-

Nicht:

einfach Tabelle ändern

Sondern:

Migration erstellen

↓

Testen

↓

Anwenden

79.2 Data Integrity

Wichtige Regeln:

- keine kaputten Beziehungen
 - keine Duplikate ohne Grund
 - Foreign Keys beachten
-

80. Git Workflow

80.1 Branches

Empfohlen:

main

develop

feature/name

bugfix/name

80.2 Commit Messages

Format:

```
type: description
```

Beispiele:

```
feat: add book upload system  
fix: repair reader progress saving  
docs: update architecture bible
```

81. Issue Workflow

Neue Funktionen:

```
Idea  
↓  
Issue  
↓  
Discussion  
↓  
Implementation  
↓  
Review
```

82. AI Coding Output Format

Wenn die KI Code erstellt:

Sie soll zuerst kurz erklären:

Änderung

Was wird gemacht?

Dateien

Welche Dateien ändern sich?

Grund

Warum diese Lösung?

Danach:

Code.

Beispiel:

Änderung:
Reader bekommt Fortschritts-Speicherung.

Betroffene Dateien:
- Reader.razor
- ReadingService.cs
- ReadingProgress.cs

Grund:
Fortschritt muss zwischen Sitzungen erhalten bleiben.

83. Debugging Rules

Bei Fehlern:

Nicht:

Problem sehen

↓

irgendeine Änderung machen

Sondern:

Fehler reproduzieren

↓

Ursache finden

↓

kleinste sinnvolle Änderung

↓

testen

84. Feature Development Rules

Jede neue Funktion benötigt:

Beschreibung

Was macht sie?

User Benefit

Warum existiert sie?

Technical Impact

Welche Systeme betrifft sie?

Future Compatibility

Passt sie zur späteren Entwicklung?

85. AI Decision Rules

Wenn mehrere Lösungen möglich sind:

Priorität:

1. Einfachheit
 2. Wartbarkeit
 3. Benutzererlebnis
 4. Performance
 5. Erweiterbarkeit
-

Nicht:

"technisch beeindruckend"

sondern:

"langfristig sinnvoll"

86. Documentation Rules

Dokumentieren:

- Architekturentscheidungen
 - wichtige Features
 - Datenmodelländerungen
 - komplexe Logik
-

Dokumentationsorte:

documentation/

README.md

Architecture.md

Features.md

Database.md

87. Quality Checklist Before Completion

Vor Abschluss einer Aufgabe prüfen:

Funktion

- ☐ Funktioniert Feature?
 - ☐ Edge Cases behandelt?
-

Code

- ☐ sauber?
 - ☐ verständliche Namen?
 - ☐ keine unnötigen Duplikate?
-

UI

- ☐ Responsive?
 - ☐ Loading State?
 - ☐ Error State?
 - ☐ Empty State?
-

Sicherheit

- ☐ Berechtigungen geprüft?
 - ☐ Eingaben validiert?
-

Zukunft

- ☐ passt Architektur?
 - ☐ keine unnötigen Einschränkungen?
-

88. Final AI Instruction

Bei jeder Entwicklung an Lumina Chronica gilt:

Baue nicht nur Funktionen. Baue eine langfristig hochwertige digitale Bibliothek.

Die Anwendung soll:

- technisch sauber
- angenehm zu benutzen
- erweiterbar
- persönlich
- atmosphärisch

sein.

Die wichtigste Priorität:

Der Nutzer soll gerne zurückkommen und das Gefühl haben, dass Lumina Chronica sein eigener digitaler Ort für Bücher, Wissen und Geschichten ist.

End of Master Project Bible